

monitoring

`monitoring`은 FARM/LAB 서버의 자원, GPU, container와 주요 service 상태를 지속적으로 수집하고 metric, dashboard와 alert로 제공한다. 서버에서 수집한 상태는 Prometheus에 저장되며, Grafana에서 조회하고 Alertmanager를 통해 운영 알림으로 전달한다.

문서	내용
설계	monitoring의 목표와 구조, 구성요소별 역할·입력·처리·출력, 주요 설정과 코드 위치
운영	exporter와 monitoring stack 배포, endpoint 확인, 장애 진단과 변경 검증 절차

monitoring 설계

개요 · 운영

GitHub 코드 링크: `admin_infra_server`는 비공개 저장소다. 링크를 누르면 GitHub 로그인 화면을 거쳐 해당 파일로 이동한다. 조직 저장소에 접근 권한이 있는 계정으로 로그인해야 한다.

1. 개요

`monitoring`의 목표는 FARM/LAB 서버와 주요 service의 현재 상태와 변화 추이를 지속적으로 관측하고, 운영자가 dashboard와 alert를 통해 이상 상태를 확인할 수 있게 하는 것이다. 관측 범위에는 CPU, memory, filesystem, network, GPU, container, Docker와 외부 연결 상태가 포함된다.

서버에서는 상태를 수집해 Prometheus metric으로 제공한다. Prometheus는 metric을 주기적으로 저장하고 alert rule을 평가하며, Grafana는 저장된 시계열을 dashboard로 표시한다.

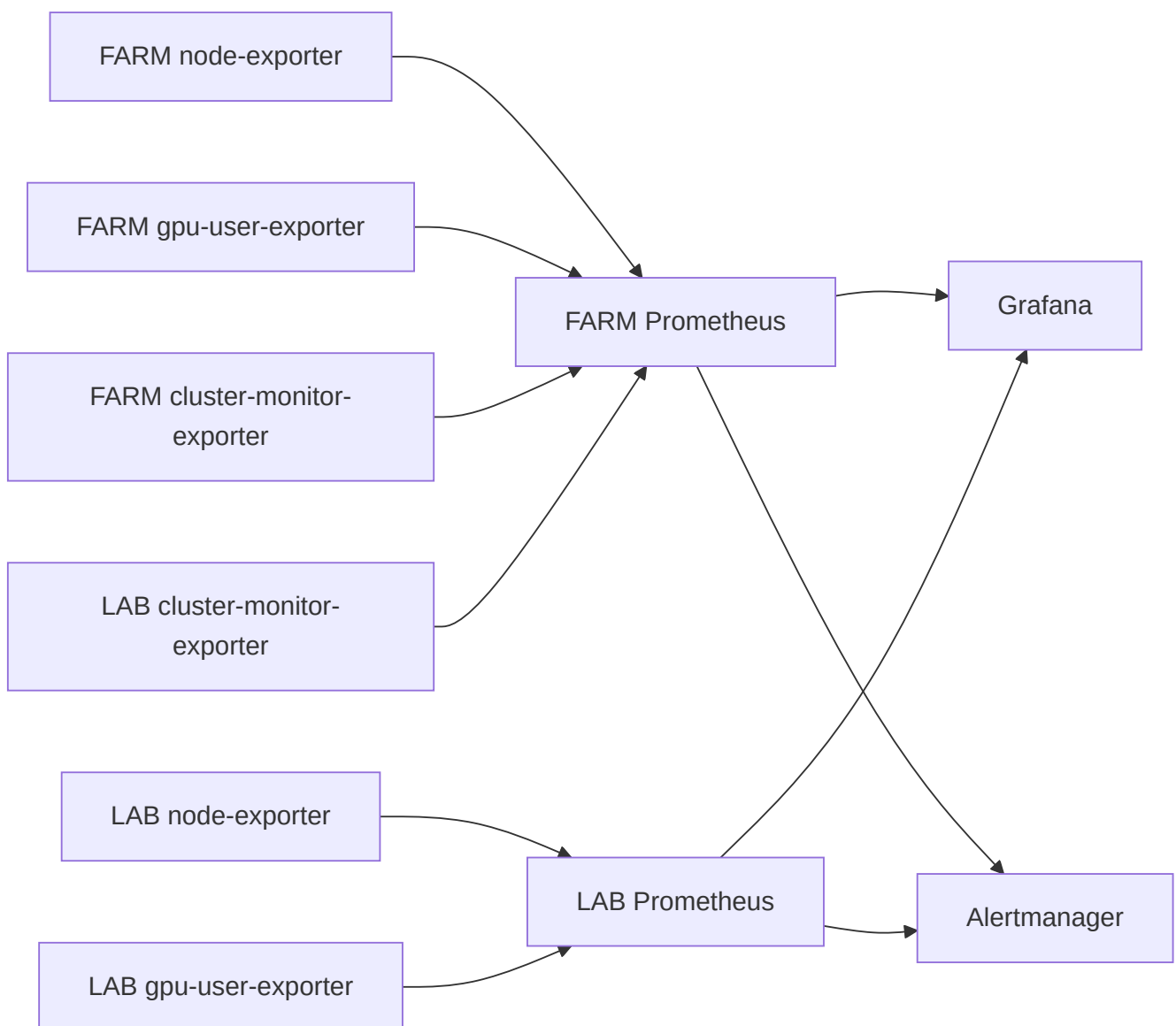
2. 설계 구조

monitoring은 서버에서 상태를 만드는 구성요소와 metric을 저장·조회·전달하는 control plane으로 구성된다.

구분	구성요소	구현 형태	역할
서버 자원 수집	<code>node-exporter</code>	Prometheus 기본 exporter	CPU, memory, filesystem, disk와 network 상태를 metric으로 제공한다.
GPU 사용량 수집	<code>gpu-user-exporter</code>	커스텀 Go exporter	GPU process를 container와 사용자 정보에 연결한다.

구분	구성요소	구현 형태	역할
운영 상태 수집	cluster-monitor-exporter	커스텀 Go exporter	Docker, container, GPU와 연결 상태를 수집한다.
Metric 저장·평가	FARM/LAB Prometheus	kube-prometheus-stack	환경별 metric을 저장하고 alert rule을 평가한다.
시각화	Grafana	kube-prometheus-stack	두 Prometheus의 시계열을 하나의 dashboard 환경에서 표시한다.
Alert 처리	Alertmanager	kube-prometheus-stack	두 Prometheus가 생성한 alert를 한곳에서 묶고 routing한다.

구성요소 사이의 주요 데이터 흐름은 다음과 같다.



FARM Prometheus는 FARM 서버의 자원·GPU metric과 FARM/LAB 전체의 cluster-monitor-exporter metric을 수집한다. 공통 service alert rule은 FARM Prometheus가 평가하고 Alertmanager로 전달한다.

LAB Prometheus는 LAB 서버의 node·GPU metric을 별도로 저장한다. 현재 LAB node·GPU metric은 Grafana 조회에 사용되고, LAB의 Docker·container 같은 운영 상태 정보는 FARM Prometheus가 LAB `cluster-monitor-exporter`를 수집해 처리한다. Grafana는 두 Prometheus를 datasource로 사용하며, 두 Prometheus는 같은 Alertmanager로 alert를 보낸다. Grafana와 Alertmanager는 각각 하나의 인스턴스로 운영한다.

3. 서버 metric 수집

3.1 node-exporter

구현 형태: Prometheus 커뮤니티에서 제공하는 기본 `node_exporter`다. `kube-prometheus-stack`의 `prometheus-node-exporter` chart가 FARM/LAB node에 DaemonSet으로 배포한다. 이 저장소는 exporter 구현 코드 대신 환경별 port, collector와 scrape 설정을 관리한다.

역할: 운영체제와 hardware의 기본 자원 사용량을 제공한다.

입력: Linux kernel이 제공하는 `/proc`, `/sys`, filesystem과 network interface 정보를 읽는다.

처리: upstream `node-exporter` collector가 CPU, memory, load, filesystem, disk I/O와 network 통계를 Prometheus 형식으로 변환한다. FARM과 LAB의 `kube-prometheus-stack`이 DaemonSet으로 각 node에 배포한다.

출력: 각 서버의 `:30070/metrics`에서 `node_cpu_*`, `node_memory_*`, `node_filesystem_*`, `node_disk_*`, `node_network_*` metric을 제공한다.

주요 설정: service port, collector option과 Prometheus 연결은 FARM/LAB Prometheus values의 `prometheus-node-exporter` 항목에서 관리한다.

관련 코드: `prometheus-farm-values.yaml`, `prometheus-lab-values.yaml`

3.2 gpu-user-exporter

구현 형태: GPU process, Docker container와 UID DB 사용자 정보를 연결하기 위해 만든 커스텀 Go exporter다. Prometheus Go client의 custom collector를 구현해 scrape 요청마다 GPU·container·사용자 정보를 수집하고 metric을 생성한다.

역할: 서버의 GPU 사용량을 실제 사용자와 container 단위로 제공한다.

입력: `nvidia-smi`의 GPU·PID·memory·utilization, `/proc/<pid>/cgroup`의 container ID, Docker inspect 결과와 UID DB의 container 사용자 정보를 사용한다.

처리: GPU process의 PID를 container에 연결하고, container record에서 사용자 정보를 조회한 뒤 GPU별 사용량을 집계한다. 실행 중인 DB 등록 container는 현재 GPU process가 없는 경우에도 값이 0인 시계열을 유지한다. 사용자 정보를 찾을 수 있는 process와 별도 관리가 필요한 process는 metric에서 구분한다.

출력: `:30072/metrics`와 `:30072/-/healthy`를 제공한다. 대표 metric은 다음과 같다.

- `docker_gpu_user_memory_used_bytes`
- `docker_gpu_user_sm_utilization_percent`
- `docker_gpu_user_process_count`

- `docker_gpu_device_utilization_percent`
- `docker_gpu_exporter_ignored_process_count`

주요 설정: server ID, DB 접속값, DB cache refresh 주기, command timeout, `nvidia-smi` 경로와 `/proc` 경로를 systemd 환경 파일로 전달한다. exporter는 host PID와 Docker 정보를 읽을 수 있는 권한으로 실행된다.

관련 코드: `main.go`, `gpu-user-exporter README`, `deploy_exporters.yml`

Go 코드 구조

`gpu-user-exporter`의 Go 구현은 `main.go`에 있다. 아래 표는 실행 흐름을 이해하는 순서로 정리했다. `main`에서 시작해 설정과 collector 구성을 확인한 뒤, 한 번의 scrape가 원천 데이터를 수집하고 사용자별 metric을 만드는 과정을 따라 가면 된다.

코드	역할
<code>main</code>	DB 연결, Prometheus registry, custom collector와 HTTP endpoint를 초기화한다.
<code>config</code> , <code>loadConfig</code> , <code>resolveDSN</code>	실행 option과 환경 변수를 읽고 server ID에 맞는 DB 연결 정보를 만든다.
<code>gpuUserCollector</code> , <code>newGPUUserCollector</code>	설정과 DB cache를 보관하고 exporter가 제공할 metric을 정의한다.
<code>Collect</code>	한 번의 scrape에서 GPU, process, container와 사용자 정보를 결합해 metric을 출력한다.
<code>ownerCache</code> 와 <code>refreshIfNeeded</code>	UID DB의 active container 정보를 일정 시간 cache하고 container ID 조회를 제공한다.
<code>collectGPUInfo</code> , <code>collectGPUProcesses</code> , <code>collectPmon</code>	<code>nvidia-smi</code> 출력을 GPU·process·utilization 구조체로 변환한다.
<code>containerIDFromPID</code>	<code>/proc/<pid>/cgroup</code> 에서 GPU process가 속한 container ID를 찾는다.
<code>runningContainerZeroKeys</code> , <code>collectRunningDockerContainers</code>	실행 중인 container와 GPU 할당을 확인해 GPU process가 없는 사용자 container에도 값이 0인 시계열을 만든다.

실행 흐름은 다음과 같다.

```
main -> config·DB·registry 초기화
      -> Prometheus scrape
      -> gpuUserCollector·Collect
      -> DB cache 갱신 + GPU/process/Docker 수집
      -> PID를 container와 사용자에게 연결
      -> 사용자·container·GPU 단위 집계
      -> Prometheus metric 출력
```

3.3 cluster-monitor-exporter

구현 형태: FARM/LAB 서버의 운영 상태와 환경별 진단 항목을 수집하기 위해 만든 커스텀 Go exporter다. background collection, metric rendering과 HTTP endpoint를 직접 구현한다.

역할: 기본 자원 metric만으로 판단하기 어려운 서버와 service의 운영 상태를 수집한다.

입력: process·kernel 상태, `nvidia-smi`, Docker와 container 상태, systemd service와 외부 연결 상태를 읽는다.

처리: background collection loop가 각 점검을 실행해 마지막 결과를 memory에 보관한다. HTTP 요청은 이 결과를 Prometheus 형식으로 출력한다. 외부 명령에는 timeout을 적용하며, 마지막 수집 시각과 허용된 stale 시간을 함께 기록한다.

주요 관측 항목은 다음과 같다.

- host GPU와 Docker daemon 상태
- 대상 container의 실행, SSH와 GPU 상태
- 외부 network 연결 상태

출력: `:30074/metrics`, `:30074/healthz` 와 서버별 public `:N89/healthz` 를 제공한다. `/healthz` 는 HTTP process 응답과 최근 collection freshness를 함께 확인한다.

제한된 복구: 설정으로 활성화한 경우 대상 container 시작, container SSH service 시작과 NVML driver/library symlink 복구를 수행한다. 복구 시도와 결과는 metric으로 기록한다.

주요 설정: 수집 주기, stale 기준, command timeout, 대상 container image pattern, public health port와 복구 기능 활성화 여부를 `/etc/default/cluster-monitor-exporter` 에서 관리한다.

관련 코드: `main.go`, `환경 설정 예시`, `cluster-monitor-exporter README`

Go 코드 구조

`cluster-monitor-exporter` 도 진입점에서 metric 제공까지의 실행 순서로 읽는다. `main` 이 설정과 HTTP server를 준비하고, background loop가 영역별 점검 결과를 metric text로 만든 뒤 HTTP handler가 마지막 결과를 제공한다.

파일·코드	역할
<code>main</code>	<code>Collector</code> , background goroutine, metric·health endpoint와 public health HTTP server를 시작한다.
<code>Config</code> , <code>loadConfig</code>	환경 파일과 환경 변수를 읽고 수집 주기, timeout, endpoint와 기능별 설정을 구성한다.
<code>Collector.run</code> , <code>collectOnce</code>	설정된 주기로 전체 수집을 실행하고 완성된 metric text와 수집 시각을 교체한다.
<code>Collector.collect</code>	영역별 collector를 순서대로 호출하고 한 번의 수집 결과를 만든다.
<code>collectHostGPU</code> 부터 <code>collectContainers</code>	host 명령과 Docker 상태를 읽고 GPU, Docker와 container metric을 생성한다.
<code>renderer</code>	metric help, type, label과 값을 Prometheus text format으로 직렬화한다.
<code>handleMetrics</code> , <code>handleHealthz</code> , <code>collectionHealth</code>	마지막 수집 결과를 <code>/metrics</code> 로 제공하고 수집 시각을 기준으로 health 상태를 응답한다.

실행 흐름은 다음과 같다.

```
main -> Config 로딩·검증
      -> Collector.run background loop
      -> collect가 영역별 collector 호출
      -> renderer가 Prometheus text 생성
      -> Collector가 마지막 결과와 수집 시각 보관
      -> /metrics와 /healthz가 저장된 결과 제공
```

Collection freshness

`up` 은 Prometheus가 exporter의 HTTP endpoint에 접속했는지를 나타낸다. 실제 점검 loop의 진행 상태는 `cluster_monitor_exporter_last_collection_timestamp_seconds` 와 `cluster_monitor_exporter_metrics_stale_after_seconds` 를 함께 사용해 판단한다. 이 구조를 통해 HTTP listener와 background collection의 상태를 각각 확인할 수 있다.

Container alert 조건

container GPU 상태는 container 실행 상태와 함께 평가한다. 정지한 container는 `running=0` , `gpu_up=0` 으로 기록되고 stopped alert의 대상이 된다. 실행 중인 container에서 GPU 점검이 실패한 경우에만 GPU alert가 발생한다.

```
cluster_monitor_container_gpu_up{job="cluster-monitor-exporter"} == 0
and on (instance, server, container, image)
cluster_monitor_container_running{job="cluster-monitor-exporter"} == 1
```

`instance` , `server` , `container` , `image` label을 함께 사용해 같은 서버와 같은 container의 시계열을 결합한다.

4. Metric 저장, 시각화와 alert

4.1 Prometheus

역할: exporter metric을 주기적으로 수집해 시계열로 저장하고 alert rule을 평가한다.

FARM 구성: FARM node·GPU metric과 FARM/LAB 전체 `cluster-monitor-exporter` metric을 수집한다. 공통 service alert rule을 평가하고 Prometheus data를 FARM local PV에 저장한다.

LAB 구성: LAB node·GPU metric을 별도 Prometheus에 저장한다. LAB control plane과 storage가 FARM과 분리되어 있다. LAB node·GPU metric은 Grafana가 LAB datasource를 통해 조회하며, LAB Prometheus가 생성한 alert는 공용 Alertmanager로 전달한다.

Grafana와 Alertmanager는 FARM control plane에 각각 하나만 배포한다. FARM Prometheus는 cluster 내부 Service를 사용하고, LAB Prometheus는 Alertmanager의 NodePort endpoint를 사용한다.

주요 설정: scrape target, label, retention, storage, alert rule과 Helm release 설정은 환경별 values에서 관리한다. FARM values가 운영 alert rule의 기준 파일이다.

관련 코드: `prometheus-farm-values.yaml` , `prometheus-lab-values.yaml` , `deploy_prometheus.yml`

4.2 Alertmanager

역할: FARM/LAB Prometheus가 생성한 alert를 한곳에서 받아 label과 상태를 기준으로 묶고 receiver로 routing한다.

입력: 두 Prometheus가 alert rule을 평가해 만든 firing·resolved alert와 각 alert의 label·annotation을 사용한다.

처리: 같은 alert를 group label에 따라 묶고 route의 matcher에 맞는 receiver를 선택한다. 상태가 resolved로 바뀌면 같은 alert group의 해제 상태도 처리한다.

출력: receiver별로 정리된 alert와 현재 firing·resolved 상태를 제공한다.

주요 설정: 공용 Alertmanager의 route, receiver, NodePort와 Secret 연결은 FARM Prometheus values에서 관리한다. LAB Prometheus의 공용 Alertmanager endpoint는 LAB Prometheus values에서 관리한다.

관련 코드: [Alertmanager 설정](#), [LAB Prometheus 연결 설정](#)

4.3 Grafana

역할: FARM과 LAB의 시계열을 서버, GPU와 network 관점의 dashboard로 제공한다.

입력: 기본 FARM datasource와 UID가 `prometheus-lab` 인 LAB datasource를 사용한다.

처리: dashboard query가 datasource, cluster, server, instance, GPU와 network interface label을 기준으로 시계열을 선택하고 비교한다.

출력: GPU usage와 network traffic dashboard를 제공한다. Grafana service는 NodePort `30080` 을 사용한다.

주요 설정: datasource와 dashboard ConfigMap은 `monitoring/grafana/` 에서, Grafana service·persistence·Secret 연결은 FARM Prometheus values에서 관리한다.

관련 코드: [Grafana dashboards](#), [LAB datasource](#), [Grafana PV](#)

5. 배포와 설정 구조

위치	역할
<code>monitoring/ansible_playbook/inventory.ini</code>	FARM/LAB 배포 대상과 SSH 접속 정보
<code>monitoring/ansible_playbook/group_vars/exporters.yml</code>	두 custom exporter의 공통·서버별 설정
<code>monitoring/ansible_playbook/deploy_exporters.yml</code>	두 custom exporter build·설치·검증
<code>monitoring/ansible_playbook/deploy_prometheus.yml</code>	FARM/LAB monitoring stack 검증·배포
<code>monitoring/prometheus/config/</code>	Prometheus, Alertmanager, PV와 storage 설정
<code>monitoring/grafana/</code>	datasource와 dashboard 원본

Ansible은 Go exporter binary를 build하고 서버별 환경 파일과 systemd unit을 설치한다. Prometheus 배포는 대상 Kubernetes context, node 상태, PV와 Secret을 확인한 뒤 환경별 Helm values를 적용한다. 구체적인 변경·배포·점검 절차는 [운영 문서](#)에서 설명한다.

6. 주요 설계 기준

6.1 HTTP 응답과 collection 상태 분리

`cluster-monitor-exporter`는 background loop의 결과를 endpoint에서 제공한다. Prometheus의 `up`, exporter의 마지막 collection 시각과 stale 기준을 함께 사용해 HTTP listener와 실제 점검 진행 상태를 각각 확인한다.

6.2 Alert label과 secret 관리

alert는 `instance`, `server`, `container`, `image`, `cluster` 처럼 진단에 필요한 label을 사용한다. credential과 password는 Kubernetes Secret과 systemd 환경 파일에서 제공하며 metric과 alert payload에는 상태와 식별 정보만 기록한다.

monitoring 운영

이 문서는 monitoring 구성 요소를 배포하고 endpoint, metric, alert와 incident를 점검하는 방법을 설명한다.

운영 원칙

- `up=1` 과 실제 collection freshness를 구분한다. HTTP process가 응답해도 마지막 성공 collection이 오래됐으면 정상으로 판단하지 않는다.
- metric 수집, 증거 보존과 상태 변경을 분리한다. exporter는 mount하지 않고, forensics는 service restart나 reboot를 수행하지 않는다.
- NFS caller가 D-state이면 `find`, `du`, `stat`, canary나 recovery를 반복 실행하지 않고 local state와 이미 수집된 snapshot을 먼저 보존한다.
- 자동 복구 결과는 원래 정상 상태와 구분해 metric과 state에 남긴다. inhibit 또는 backoff가 설정된 worker를 endpoint 호출로 우회하지 않는다.
- 배포는 `monitoring/ansible_playbook/` 을 기준으로 하고 target host의 개별 install script를 운영 entry point로 사용하지 않는다.
- DB password, Grafana password, webhook과 Kerberos credential은 metric, alert label, Git values와 일반 journal에 기록하지 않는다.

1. 배포 순서

새 host 또는 NFS GSS monitoring을 처음 적용할 때는 다음 순서를 권장한다.

1. inventory와 host FQDN/SPN/mount source 확인

2. NFS GSS readiness/canary/recovery worker 설치
3. NFS forensics 설치
4. exporter 배포
5. Prometheus target/rule 반영
6. Grafana datasource/dashboard 확인
7. alert delivery test

운영 배포 entry point는 exporter 내부 개별 shell script가 아니라 `monitoring/ansible_playbook/` 이다.

2. Exporter 배포

```
cd /home/jy/server_manage/monitoring
```

두 exporter를 한 host에 배포:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_exporters.yml \  
-e exporter_hosts=farm8
```

FARM 전체 또는 FARM/LAB 전체:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_exporters.yml \  
-e exporter_hosts=FARM
```

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_exporters.yml
```

한 exporter만 전체 재배포:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_cluster_monitor_exporter_all.yml
```

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_gpu_user_exporter_all.yml
```

target에는 다음 systemd service와 endpoint가 생긴다.

```
cluster-monitor-exporter.service -> :30074/metrics, :30074/healthz, :N89/healthz
gpu-user-exporter.service        -> :30072/metrics, :30072/-/healthy
```

배포 기준과 요구 조건은 [Ansible README](#), 실제 task는 [deploy_exporters.yml](#)에서 확인한다.

3. NFS GSS와 forensics 배포

NFS GSS readiness/canary/recovery worker:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_nfs_gss_health.yml \
-e nfs_gss_health_hosts=lab8
```

NFS forensics:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_nfs_forensics.yml
```

중요한 systemd unit과 state:

```
decs-kerberos-nfs-ready.service
decs-nfs-gss-canary-probe.service/.timer
decs-nfs-gss-mount-recovery.service/.timer
/run/decs-nfs-gss/ready.state
/var/lib/decs-nfs-gss/canary.state
/var/lib/decs-nfs-gss/recovery.state

decs-nfs-forensics-buffer.service
decs-nfs-forensics-watch.service/.timer
decs-nfs-forensics-snapshot.service
/run/decs-nfs-forensics/status.json
/var/lib/decs-nfs-forensics/
```

FARM은 전용 read-only canary export가 준비될 때까지 canary만 비활성화하며 keytab/kinit/kvno/rpc-gssd readiness와 guarded recovery는 유지한다. LAB canary는 `sec=krb5` 전용 read-only export를 사용한다.

4. Keytab checker 배포

FARM 읽기 전용 checker:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_kerberos_nfs_keytab_check.yml
```

LAB을 상시 점검 대상으로 전환한 뒤에만 두 profile을 활성화한다.

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_kerberos_nfs_keytab_check.yml \  
-e '{"keytab_check_profiles":["farm","lab"]}'
```

```
systemctl --user list-timers 'check-nfs-keytab@*.timer'  
systemctl --user status check-nfs-keytab@farm.service --no-pager
```

checker는 drift를 고치지 않는다. KVNO history, repair와 rotation은 [Kerberos/NFS 운영](#)을 따른다.

5. Prometheus/Alertmanager 배포

먼저 server-side render와 cluster precondition만 검증한다.

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_prometheus.yml
```

변경 적용:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_prometheus.yml \  
-e prometheus_dry_run=false
```

한 환경만 배포하려면 `prometheus_farm_enabled=false` 또는 `prometheus_lab_enabled=false` 를 사용한다. playbook은 kubeconfig 대상 cluster, node Ready/DiskPressure, 필수 Secret 이름을 확인한 후 Helm을 실행한다.

FARM에서 필요한 Secret:

- `monitoring-grafana-admin`: `admin-user`, `admin-password`
- `cluster-monitor-slack-webhook-farm`: `url`

값은 Git values에 넣지 않는다. Alertmanager는 Slack webhook으로 직접 보내지 않고 localhost relay를 거쳐 internal notify API로 보낸다.

현재 control plane의 역할은 다음처럼 구분한다.

- FARM Prometheus는 FARM 자원 metric과 FARM/LAB의 `cluster-monitor-exporter` 를 수집하고 중앙 alert rule을 평가한다.

- Alertmanager는 하나만 실행하며 FARM/LAB Prometheus가 생성한 alert를 함께 처리한다. LAB Prometheus는 `192.168.2.199:30903`으로 연결한다.
- LAB Prometheus는 LAB node/GPU metric을 독립 저장한다.
- Grafana는 하나만 실행하며 기본 FARM datasource와 `prometheus-lab` datasource를 함께 사용한다.

6. 배포 후 endpoint 확인

target host에서:

```
systemctl status cluster-monitor-exporter gpu-user-exporter --no-pager
journalctl -u cluster-monitor-exporter -n 100 --no-pager
journalctl -u gpu-user-exporter -n 100 --no-pager

curl -fsS http://127.0.0.1:30074/healthz
curl -fsS http://127.0.0.1:30074/metrics | head
curl -fsS http://127.0.0.1:30072/-/healthy
curl -fsS http://127.0.0.1:30072/metrics | head
curl -fsS http://127.0.0.1:30070/metrics | head
```

:30070의 `node-exporter`는 `kube-prometheus-stack`에서 배포하므로 앞의 두 custom exporter처럼 host systemd unit을 확인하지 않는다. endpoint가 없으면 그 cluster의 DaemonSet, Service와 node scheduling 상태를 확인한다.

public health port는 서버 번호 `N`에 대해 `9000 + N*100 - 11`이다. 예를 들어 FARM8은 `9789/healthz`다. 이 endpoint는 exporter process/collection freshness와 외부 NAT 도달 경로를 확인하는 용도이며, 모든 dependency가 준비됐음을 보장하지는 않는다.

NFS GSS host-only API:

```
curl -fsS http://127.0.0.1:30074/nfs-gss/ready
curl -fsS http://127.0.0.1:30074/nfs-gss/health
curl -fsS -X POST http://127.0.0.1:30074/nfs-gss/probe
curl -fsS -X POST http://127.0.0.1:30074/nfs-gss/recover
```

`probe`와 `recover`는 loopback 요청만 허용하며 worker를 queue한 뒤 즉시 응답한다. `recover`를 반복 호출하기 전에 recovery state와 inhibit reason을 확인한다.

7. 무엇을 어디에서 보는가

관측 대상	대표 metric/alert	구현/설정 링크
CPU, memory, filesystem, disk, network	<code>node_cpu_*</code> , <code>node_memory_*</code> , <code>node_filesystem_*</code> , <code>node_disk_*</code> , <code>node_network_*</code>	FARM values, LAB values
exporter process와 freshness	<code>up</code> , <code>cluster_monitor_exporter_last_collection_timestamp_seconds</code> , <code>ClusterMonitorExporter*</code>	collector, rules
required mount와 latency	<code>cluster_monitor_host_mount_up</code> , <code>..._responsive</code> , <code>..._probe_seconds</code>	mount collector, storage dashboards
Kerberos NFS readiness	<code>cluster_monitor_nfs_gss_ready</code> , <code>ClusterMonitorNFSGSSReadinessFailed</code>	readiness, GSS collector
canary/recovery	<code>cluster_monitor_nfs_gss_canary_*</code> , <code>..._recovery_*</code>	canary, recovery
D-state, lease, hung task	<code>cluster_monitor_host_dstate_*</code> , <code>cluster_monitor_nfs_*</code> , <code>ClusterMonitorNFS*</code>	forensics adapter, forensics scripts
host GPU/Docker/container	<code>cluster_monitor_host_gpu_up</code> , <code>...docker_daemon_up</code> , <code>...container_*</code>	cluster collector
사용자별 GPU	<code>docker_gpu_user_memory_used_bytes</code> , <code>...sm_utilization_percent</code> , <code>...process_count</code>	gpu-user-exporter, GPU dashboard
외부 연결	<code>cluster_monitor_external_connectivity_*</code> , public health	exporter, Apps Script
AD/NAS keytab	checker JSON/exit code	keytab checker

canonical alert rule은 [prometheus-farm-values.yaml](#)이다. exporter 내부

`prometheus/cluster_monitor_alerts.yml` 은 metric 이름을 보여 주는 reference이며 실제 FARM release와 값이 다를 수 있다.

8. Alert별 진단 순서

Exporter down 또는 stale

1. Prometheus target에서 connection error와 scrape timestamp를 구분한다.
2. target systemd status/journal을 확인한다.
3. `:30074/healthz` 와 `/metrics` 를 각각 확인한다.
4. process는 살아 있지만 stale이면 마지막 실행 command와 D-state를 확인한다.
5. 변경된 env/config와 binary 배포 시간을 확인한 뒤 exporter만 재배포한다.

Required mount down/slow/unresponsive

1. `findmnt` 로 mount 존재, FQDN source, filesystem과 `sec` 를 확인한다.
2. mount probe와 storage ping/peer diagnosis label을 확인한다.
3. GSS readiness에서 처음 실패한 stage를 확인한다.
4. D-state, lease expiry, hung-task와 local forensic snapshot을 확인한다.
5. mount가 없을 때만 recovery state를 확인한다.
6. mount가 이미 있거나 D-state caller가 있으면 강제 remount를 반복하지 않는다.

Kerberos source는 IP가 아니라 FQDN이어야 한다. 자세한 기준은 [Kerberos/NFS 운영의 mount 절](#)을 따른다.

GSS readiness/canary/recovery

```
systemctl status decs-kerberos-nfs-ready.service --no-pager
systemctl status decs-nfs-gss-canary-probe.timer --no-pager
systemctl status decs-nfs-gss-mount-recovery.timer --no-pager
journalctl -u decs-kerberos-nfs-ready.service -n 100 --no-pager
cat /run/decs-nfs-gss/ready.state
cat /var/lib/decs-nfs-gss/canary.state
cat /var/lib/decs-nfs-gss/recovery.state
```

- keytab/kinit stage: host machine credential 확인
- kvno stage: DNS/FQDN/SPN/KDC 확인
- rpc-gssd stage: service와 잘못된 `-n` override 확인
- canary stage: 전용 export와 user share를 혼동하지 않았는지 확인
- inhibited recovery: D-state 또는 mount timeout 증거를 보존하고 원인을 처리

GPU 사용자 mapping 이상

1. `nvidia-smi` 와 exporter scrape success를 확인한다.
2. ignored process count가 증가했는지 확인한다.
3. PID의 cgroup container ID와 Docker inspect를 비교한다.
4. DB cache refresh success와 cache age/entry를 확인한다.
5. UID DB의 container record와 실제 running container를 비교하고 DB 갱신 절차로 수정한다.

Container SSH/GPU alert

1. container가 running인지 먼저 확인한다.
2. exporter가 start/SSH/NVML recovery를 시도했는지 metric과 journal을 확인한다.
3. container image가 configured regex 대상인지 확인한다.
4. NVML mismatch가 아니라 application/GPU allocation 문제면 자동 symlink repair를 반복하지 않는다.

9. Grafana 점검

- LAB dashboard datasource UID가 `prometheus-lab` 인지 확인한다.
- dashboard query의 `cluster`, `server_id`, `job` label이 scrape config와 일치하는지 확인한다.
- storage dashboard에서 mount probe, responsive, D-state, blocked process, hung-task를 같은 시간축으로 비교한다.
- GPU dashboard에서 DB-known 사용자 metric과 device 전체 metric을 함께 본다.
- 새 server 추가 시 dashboard에 hard-coded server/GPU panel 누락이 없는지 확인한다.

dashboard 원본:

- [FARM storage latency](#)
- [LAB storage latency](#)
- [GPU usage](#)
- [Network traffic](#)

10. 테스트

```
cd /home/jy/server_manage/monitoring

(cd prometheus/exporters/cluster-monitor-exporter && go test ./...)
(cd prometheus/exporters/gpu-user-exporter && go test ./...)
bash nfs-forensics/tests/test_watch.sh
bash prometheus/exporters/cluster-monitor-exporter/tests/test_nfs_gss_ready.sh
bash prometheus/exporters/cluster-monitor-exporter/tests/test_nfs_gss_mount_recovery.sh
python3 prometheus/config/tests/test_slack_notify_relay.py
```

변경 위험에 맞게 최소한 다음을 함께 검증한다.

- metric 이름/type/help와 0/failure path
- recovery가 healthy mount를 건드리지 않는지
- D-state에서 mount attempt가 차단되지만 existing mount는 healthy로 처리되는지
- Alertmanager relay가 secret을 출력하지 않고 internal API payload로 변환하는지
- FARM/LAB dashboard datasource가 바뀌지 않았는지

11. 새 서버 추가 체크리스트

1. `ansible_playbook/inventory.ini` 에 `farmN` / `labN` 형식으로 추가
2. `group_vars/exporters.yml` 의 mount source, SPN과 환경 기본값 확인
3. host share target와 public `N89` port 계산 확인

4. GSS readiness/forensics/exporter 배포
5. Prometheus `:30070`, `:30072`, `:30074` scrape target 추가
6. 방화벽/NAT public health 확인
7. Grafana dashboard server/GPU panel 확인
8. alert rule label과 Alertmanager route 확인
9. endpoint와 실제 alert delivery 검증

12. 운영 문서에 계속 추가할 내용

설계와 운영을 분리한 뒤 다음 정보를 운영 문서에 누적하면 좋다.

- 서비스별 owner, endpoint, systemd/Kubernetes resource와 dependency
- metric별 정상 범위, alert `for` 와 runbook link
- 최근 배포 version/commit과 마지막 검증 시각
- FARM/LAB retention, PV 용량과 capacity threshold
- alert silence 승인/만료 기준과 false-positive 기록
- 자동 복구별 시도 횟수, inhibit 해제와 rollback 절차
- incident snapshot 보존 기간, 접근 권한과 개인정보 취급
- 새 server/metric/dashboard 추가 checklist

설계 문서에는 구성 요소의 관계와 안전 경계를 유지하고, host별 현재 값과 일회성 incident 결과는 canonical config/runbook 또는 별도 evidence에 기록한다.